

# Algorithm employed to extract various shape descriptors from 3D Objects.

César Eduardo Muñoz-Chávez      Hermilo Sánchez-Cruz  
Elisa Isabel Gómez-Gómez

February 2, 2025

## 1 Appendix D. Shape Descriptor Extraction.

This algorithm iterates over every voxel in a 3D binary image to compute two key shape descriptors: the volume (total number of 1-voxels) and the enclosing surface (sum of exposed faces of each voxel). The algorithm works as follows:

- It first verifies that the input is a 3D binary image.
- It initializes counters for volume and enclosing surface.
- It iterates over each voxel and checks its six possible neighbors.
- For each 1-voxel, it increments the volume counter.
- It counts the number of exposed faces by checking if each neighbor is a 0 (or if the voxel is on the boundary of the image).
- The enclosing surface count accumulates all exposed faces.

This method efficiently quantifies the shape of voxelized objects, aiding in their topological analysis. The algorithm employed to extract key shape descriptors, specifically the enclosing surface and volume, as detailed in Algorithm 1.

---

**Algorithm 1** Calculate Volume and Enclosing Surface of 1-Voxels Regions in a 3D Binary Image

---

**Require:** 3D binary image Image where 1 represents the region of interest

**Ensure:** Total volume and enclosing surface of the 1-pixel regions in the 3D image

```

1: Initialize Count_ones  $\leftarrow 0$ 
2: Initialize Enclosing_surface  $\leftarrow 0$ 
3: if Image.ndim  $\neq 3$  then
4:   raise ValueError("Input should be a 3D binary image.")
5: end if
6: for  $i \leftarrow 0$  to Image.shape[0] - 1 do
7:   for  $j \leftarrow 0$  to Image.shape[1] - 1 do
8:     for  $k \leftarrow 0$  to Image.shape[2] - 1 do
9:       if Image[ $i, j, k$ ] = 1 then
10:        Count_ones  $\leftarrow$  Count_ones + 1
11:        Initialize neighbors  $\leftarrow 0$  {Initialize the neighbor count}
12:        if  $i = 0$  or Image[ $i - 1, j, k$ ] = 0 then
13:          neighbors  $\leftarrow$  neighbors + 1
14:        end if
15:        if  $i =$  Image.shape[0] - 1 or Image[ $i + 1, j, k$ ] = 0 then
16:          neighbors  $\leftarrow$  neighbors + 1
17:        end if
18:        if  $j = 0$  or Image[ $i, j - 1, k$ ] = 0 then
19:          neighbors  $\leftarrow$  neighbors + 1
20:        end if
21:        if  $j =$  Image.shape[1] - 1 or Image[ $i, j + 1, k$ ] = 0 then
22:          neighbors  $\leftarrow$  neighbors + 1
23:        end if
24:        if  $k = 0$  or Image[ $i, j, k - 1$ ] = 0 then
25:          neighbors  $\leftarrow$  neighbors + 1
26:        end if
27:        if  $k =$  Image.shape[2] - 1 or Image[ $i, j, k + 1$ ] = 0 then
28:          neighbors  $\leftarrow$  neighbors + 1
29:        end if
30:        Enclosing_surface  $\leftarrow$  Enclosing_surface + neighbors {Add to total surface area}
31:      end if
32:    end for
33:  end for
34: end for
35: return Count_ones, Enclosing_surface

```

---